

CI/CD 인증 보안 강화를 위한 Google Cloud Workload Identity Federation 아키텍처 종합 검토 및 전략적 권고안

섹션 1: 제출된 WIF 인증 문제 보고서에 대한 심층 평가

본 섹션에서는 제공된 내부 보고서(이하 '기존 보고서')에 대한 전문가 수준의 평가를 제공합니다. 기존 보고서의 분석 내용 중 정확한 부분을 검증하고, 더 깊이 있는 기술적 맥락을 부여하며, 제안된 전략의 잠재적 위험을 평가하여 보다 성숙한 아키텍처로 나아가기 위한 기반을 마련하는 것을 목표로 합니다.

1.1. WIF 문제 해결 체크리스트 분석

기존 보고서에서 제시한 Workload Identity Federation (WIF) 문제 해결 체크리스트는 `invalid_target` 오류 발생 시 점검해야 할 핵심 구성 요소들을 정확하게 명시하고 있습니다.¹ 체크리스트 항목인 프로젝트 번호, 워크로드 아이덴티티 풀 및 공급자(Provider)의 상태, 서비스 계정 바인딩, GitHub 시크릿 값, 그리고 워크플로 파일 경로는 실제로 WIF 인증 실패의 가장 일반적인 원인들입니다. 이 오류는 Google Cloud의 Security Token Service (STS)가 GitHub Actions로부터 받은 OIDC(OpenID Connect) 토큰의 클레임(claim)을 기반으로 일치하고 활성화된 공급자를 찾지 못할 때 발생합니다.²

이 체크리스트의 각 항목은 단순히 독립된 설정 값이 아니라, STS에 대한 단일 API 호출을 구성하는 상호 연결된 매개변수 집합이라는 점을 이해하는 것이 중요합니다. GitHub Actions의 `google-github-actions/auth` 액션은 워크플로 실행 중에 GitHub으로부터 OIDC 토큰을 발급받습니다. 이 토큰에는 실행된 리포지토리(repository), 브랜치(ref) 등의 정보가 클레임 형태로 포함되어 있습니다. 이 액션은 해당 토큰과 워크플로 파일에 명시된 `workload_identity_provider` 경로 같은 입력 값을 사용하여 GCP의 STS에 연동 토큰(federated token)을 요청하는 API 호출을 생성합니다.² STS는 이 요청에 포함된 공급자 경로를 사용하여 WIF 공급자 구성을 조회합니다. 만약 이 경로가 잘못되었거나, 경로에 포함된 프로젝트 번호가 다르거나, 해당 공급자가 비활성화(DISABLED) 상태라면, STS는 토큰 교환을 위한 '대상(target)'이 유효하지 않다고 판단하여 `invalid_target` 오류를

반환합니다.

여기서 `invalid_target` 오류 메시지가 의도적으로 일반화되어 있다는 점을 주목해야 합니다. 이는 잠재적인 공격자에게 클라우드 환경의 내부 구성(예: "프로젝트 번호 12345는 존재하지만, 'gha-provider'라는 이름의 공급자는 없습니다")에 대한 구체적인 정보를 노출하지 않기 위한 보안 조치입니다. 따라서 기존 보고서의 체크리스트는 이러한 "블랙박스" 환경에서 문제를 진단하기 위한 필수적인 과정이지만, 각 항목이 STS 토큰 교환 프로세스에서 어떤 역할을 하는지 이해하면 단순한 점검 목록을 넘어, 문제의 근본 원인을 추적하는 진단 맵(diagnostic map)으로 활용될 수 있습니다.

1.2. 서비스 계정 키 대체 전략 평가

기존 보고서는 WIF 문제 해결이 지연될 경우, 배포를 재개하기 위한 임시 방편으로 서비스 계정의 JSON 키를 사용하는 대체 전략을 제안합니다.¹ 이 방법은 `credentials_json` 입력을 통해 WIF 프로세스를 완전히 우회하고 서비스 계정으로 직접 인증하므로, 배포 차단을 해소하는 데는 즉각적으로 효과적입니다. 그러나 이 전략은 WIF가 제공하는 핵심적인 보안 이점, 즉 수명이 긴(long-lived) 내보내기 가능한 키의 필요성을 제거한다는 원칙과 정면으로 배치됩니다.⁴

서비스 계정 키를 임시방편으로 사용하는 것은 상당한 "보안 부채(security debt)"를 발생시키는 행위입니다. 이 부채는 시간이 지남에 따라 조직의 보안 리스크를 지속적으로 증가시킵니다. GitHub 시크릿에 저장된 서비스 계정 키(GCP_SA_KEY)는 그 자체로 높은 가치를 지닌 공격 대상이 됩니다. 이 키가 존재하는 기간이 길어질수록, 실수로 인한 노출, 내부자 위협, 또는 GitHub 저장소 자체의 보안 침해 사고를 통해 탈취될 위험이 기하급수적으로 증가합니다. 서비스 계정 키는 정적인 자격 증명으로, 한번 탈취되면 공격자는 해당 서비스 계정에 부여된 모든 권한을 GCP 내에서 자유롭게 행사할 수 있게 됩니다.

반면, WIF는 각 CI/CD 파이프라인 실행마다 동적으로 생성되고 단 한 번의 토큰 교환에만 사용되는, 수명이 매우 짧은(short-lived) OIDC 토큰을 기반으로 동작합니다.² 이는 자격 증명이 노출되더라도 그 유효 기간이 매우 짧아 공격에 활용될 가능성이 현저히 낮은, 훨씬 우월한 보안 모델을 제공합니다. 따라서 서비스 계정 키를 사용하는 대체 전략은 WIF 디버깅이라는 일시적인 불편함을 회피하기 위해, 지속적이고 파급력이 큰 보안 취약점을 감수하는 행위입니다. 이 트레이드오프는 조직 내에서 명확하게 인지되어야 하며, 해당 키를 언제까지 폐기할 것인지에 대한 엄격한 기한이 설정된 구체적인 원복 계획과 함께 관리되어야 합니다.

1.3. 장기적 WIF 재구성 전략 검토

기존 보고서에서 제안한 장기적 WIF 재구성 전략—워크로드 아이덴티티 풀 및 공급자 재정의, 서비스 계정 권한 최소화, 정기적 검증, 그리고 임시 키 폐기—은 개념적으로 매우 올바르며 Google Cloud의

보안 모범 사례와도 일치합니다.¹ 특히, 리포지토리나 브랜치 같은 속성을 기반으로 접근을 제한하는 attribute-condition의 사용과 최소 권한 원칙(principle of least privilege)의 적용은 안전한 WIF 구성을 위한 핵심 요소입니다.

그러나 이 전략은 치명적인 약점을 가지고 있는데, 바로 실행 방법론의 부재입니다. 보고서는 gcloud CLI 명령어를 사용하여 이러한 구성을 수동으로 적용할 것을 암시하고 있습니다. 이는 현재 발생한 invalid_target 오류의 근본 원인이었을 가능성이 높은 접근 방식, 즉 수동 구성으로 인한 설정 불일치(configuration drift)와 인적 오류(human error)의 문제를 해결하지 못합니다. 수동 명령어 기반의 접근 방식은 확장성이 부족하고, 변경 이력을 추적하기 어려우며, 감사(auditing)에 취약합니다.

진정한 의미의 장기적이고 안정적인 해결책은 '무엇을' 구성할 것인가를 넘어, '어떻게' 구성하고 관리할 것인가에 대한 아키텍처적 전환을 포함해야 합니다. 현재의 문제는 '의도된 상태(working WIF)'와 '실제 상태(misconfigured WIF)' 간의 불일치에서 비롯되었습니다. gcloud 명령어로 이를 수정하는 것은 단순히 반응적인 조치에 불과합니다. 보다 근본적인 해결책은 Terraform과 같은 코드형 인프라(Infrastructure as Code, IaC) 도구를 도입하여, WIF 구성을 포함한 모든 인프라의 '의도된 상태'를 코드로 정의하고 관리하는 것입니다.⁵ 이 코드는 버전 관리 시스템(예: Git)을 통해 관리되고, 동료 검토(peer review)를 거치며, 자동화된 파이프라인을 통해 일관성 있게 적용됩니다. 이러한 선언적(declarative) 접근 방식은 설정 불일치의 가능성을 원천적으로 차단하고, 전체 인프라 구성의 투명성과 재현성을 보장합니다. 따라서 기존 보고서의 전략은 훌륭한 목표를 제시하지만, IaC라는 핵심적인 실행 방법론이 결여된 불완전한 제안이라고 할 수 있습니다.

섹션 2: invalid_target 오류에 대한 고급 진단 및 검증 기법

invalid_target 오류를 해결하기 위해 단순히 체크리스트에 의존하는 추측성 접근법을 넘어, Google Cloud가 제공하는 네이티브 도구를 활용하여 증거 기반의 정밀 진단을 수행하는 고급 기법을 제시합니다. 이를 통해 문제 해결 시간을 단축하고 재발을 방지할 수 있습니다.

2.1. Cloud Audit Logs를 활용한 세분화된 오류 분석

Google Cloud는 IAM 및 STS API 활동에 대한 상세한 감사 로그를 제공하여, WIF 인증 과정의 모든 단계를 투명하게 추적할 수 있는 강력한 기능을 제공합니다.¹⁰ invalid_target와 같은 WIF 인증 실패 문제를 정밀하게 진단하기 위해서는, 먼저 **Security Token Service API**에 대한 **데이터 액세스(Data Access) 감사 로그**를 활성화해야 합니다. 이 로그를 활성화하면 성공적인 토큰 교환뿐만 아니라 실패한 모든 시도에 대한 상세 정보가 기록됩니다.²

인증 실패 시 생성되는 감사 로그 항목에는 문제 해결에 결정적인 단서를 제공하는 여러 필드가

포함됩니다. 가장 중요한 필드는 다음과 같습니다:

- `protoPayload.authenticationInfo.principalSubject`: 외부 ID 공급자(IdP), 즉 GitHub Actions가 발급한 OIDC 토큰의 주체(subject)를 나타냅니다. 이 값은 IAM 바인딩의 `principalSet`과 비교하여 일치 여부를 확인할 때 사용됩니다.
- `protoPayload.resourceName`: 인증 시도가 이루어진 대상 WIF 공급자의 전체 리소스 이름을 보여줍니다. 워크플로 파일에 설정된 `workload_identity_provider` 경로가 올바른지 검증하는 데 사용됩니다.
- `protoPayload.status.message` 또는 `error_description`: GitHub Actions 로그에 표시되는 일반적인 `invalid_target` 메시지보다 훨씬 구체적인 오류 설명을 포함하는 경우가 많습니다. 예를 들어, "Provider is disabled" 또는 "Pool not found"와 같이 문제의 원인을 직접적으로 지목하는 메시지를 통해 디버깅의 방향을 명확히 할 수 있습니다.³

이러한 감사 로그를 활용하면 WIF 디버깅 패러다임이 근본적으로 바뀝니다. 기존의 방식이 '체크리스트의 다섯 항목 중 무엇이 틀렸을까?'를 하나씩 변경하며 확인하는 느리고 비효율적인 소거법이었다면, 감사 로그를 활용하는 방식은 다음과 같은 직접적인 조사 워크플로를 따릅니다:

1. GCP 프로젝트에서 STS API에 대한 데이터 액세스 감사 로그('Admin Read' 및 'Data Read')가 활성화되어 있는지 확인합니다.¹⁰
2. 오류가 발생하는 GitHub Actions 워크플로를 다시 실행하여 실패 로그 항목을 의도적으로 생성합니다.
3. GCP 콘솔의 Logs Explorer로 이동하여 다음 쿼리를 사용하여 실패한 STS 토큰 교환 요청을 필터링합니다:
`protoPayload.serviceName="sts.googleapis.com"`
`protoPayload.methodName="google.identity.sts.v1.SecurityTokenService.Token"`
`severity=ERROR`
4. 검색된 로그 항목의 `protoPayload`를 상세히 검토합니다. 여기에 기록된 `resourceName`과 `principalSubject`를 현재 WIF 구성과 비교하여 불일치 지점을 정확히 찾아냅니다. 이는 추측이 아닌 데이터에 기반한 명확한 수정 조치를 가능하게 합니다.

2.2. IAM Policy Troubleshooter 활용

IAM Policy Troubleshooter는 특정 주체(principal)가 특정 리소스에 대해 특정 권한을 가지고 있는지 여부를 진단하는 데 사용되는 강력한 GCP 도구입니다.¹² 이 도구는 WIF 인증 과정에서 발생하는 문제를 두 단계로 나누어 분석하는 데 매우 유용합니다.

WIF 인증 실패는 크게 두 가지 유형으로 나눌 수 있습니다:

1. **인증(Authentication) 실패**: GitHub Actions의 OIDC 토큰을 GCP 연동 토큰으로 교환하는 첫 단계에서 발생하는 실패입니다. `invalid_target` 오류가 바로 여기에 해당합니다.
2. **승인(Authorization) 실패**: 토큰 교환은 성공적으로 이루어졌으나, 발급받은 연동 토큰을 사용하여 서비스 계정을 위장(impersonate)하고 실제 GCP 리소스(예: Cloud Run 서비스)에

API를 호출할 때 권한 부족으로 발생하는 실패입니다. 이 경우 "Permission Denied"와 같은 오류가 발생합니다.

기존 보고서는 첫 번째 유형의 실패에만 초점을 맞추고 있습니다. IAM Policy Troubleshooter는 두 번째 유형의 문제를 해결하는 데 핵심적인 역할을 합니다. 현재 이 도구는 WIF 주체(principalSet)를 직접 지원하지 않고 사용자 또는 서비스 계정 이메일 주소를 입력받도록 되어 있지만¹², WIF가 위장하려는 **서비스 계정**의 권한을 검증하는 데 완벽하게 활용될 수 있습니다.

invalid_target 오류를 해결한 후, gcloud run deploy 명령어 실행 중에 "Permission Denied" 오류가 발생했다고 가정해 봅시다. 이 시점에서 문제의 원인은 더 이상 WIF 구성이 아니라, 위장된 서비스 계정(gha-deploy-sa@...)에 부여된 IAM 역할에 있습니다. 개발자는 IAM Policy Troubleshooter를 사용하여 이 문제를 체계적으로 진단할 수 있습니다.

1. Policy Troubleshooter 페이지로 이동합니다.
2. '주체(Principal)' 필드에 위장하려는 서비스 계정의 이메일 주소(gha-deploy-sa@quali-journal-prod.iam.gserviceaccount.com)를 입력합니다.
3. '리소스(Resource)' 필드에 배포 대상 Cloud Run 서비스의 전체 리소스 이름을 입력합니다 (예: //run.googleapis.com/projects/quali-journal-prod/locations/asia-northeast3/services/quali-journal-backend).
4. '권한(Permission)' 필드에 배포에 필요한 특정 권한(예: run.services.update)을 입력합니다.
5. '액세스 확인(Check access)'을 클릭하면, 해당 서비스 계정이 지정된 리소스에 대해 해당 권한을 가지고 있는지, 만약 있다면 어떤 역할과 정책 바인딩에 의해 부여되었는지 상세한 분석 결과를 보여줍니다.¹⁴

이 과정을 통해 문제의 원인이 인증 계층(WIF)에 있는지, 아니면 승인 계층(IAM)에 있는지를 명확하게 분리하여 진단할 수 있으며, 이는 복잡한 클라우드 환경에서 매우 효율적인 문제 해결 전략입니다.

섹션 3: 견고하고 확장 가능한 대안: 코드형 인프라(IaC)를 통한 WIF 관리

본 섹션에서는 수동 구성 방식의 근본적인 한계를 극복하고, 장기적으로 안정적이고 안전한 CI/CD 파이프라인을 구축하기 위한 결정적인 대안으로 Terraform을 활용한 코드형 인프라(IaC) 접근법을 제시합니다.

3.1. IaC 도입의 전략적 필요성

GCP 콘솔이나 gcloud CLI를 통한 수동적인 인프라 구성은 초기에는 빠르고 직관적으로 보일 수 있으나,

시스템의 복잡성이 증가함에 따라 심각한 운영상의 문제를 야기합니다. 이러한 방식은 인적 오류에 취약하고, 변경 사항에 대한 명확한 감사 추적(audit trail)을 남기기 어려우며, 여러 환경(개발, 스테이징, 프로덕션)에 걸쳐 일관성을 유지하기가 거의 불가능합니다.

Terraform과 같은 IaC 도구는 이러한 문제에 대한 근본적인 해결책을 제공합니다. IaC는 복잡한 IAM 및 WIF 구성을 포함한 모든 클라우드 인프라를 사람이 읽고 이해할 수 있는 코드(HCL - HashiCorp Configuration Language)로 정의합니다.⁵ 이 코드는 인프라의 유일한 진실 공급원(single source of truth) 역할을 하며, 다음과 같은 전략적 이점을 제공합니다.

- **재현성 및 일관성:** 동일한 Terraform 코드는 언제나 동일한 인프라 구성을 보장합니다. 이를 통해 '내 컴퓨터에서는 됐는데'와 같은 문제를 원천적으로 방지하고, 모든 환경에 걸쳐 구성의 일관성을 유지할 수 있습니다.
- **버전 관리 및 감사:** 모든 인프라 구성 변경은 Git과 같은 버전 관리 시스템을 통해 추적됩니다. 누가, 언제, 무엇을, 왜 변경했는지에 대한 완전한 이력이 남기 때문에 감사 요구사항을 충족하고 문제 발생 시 원인을 신속하게 파악할 수 있습니다.
- **코드 리뷰 및 협업 (GitOps):** 인프라 변경은 애플리케이션 코드 변경과 동일한 프로세스, 즉 풀 리퀘스트(Pull Request)와 동료 검토(Peer Review)를 통해 이루어집니다. 이는 잠재적인 오류나 보안 취약점을 사전에 발견하고, 팀 전체의 지식 공유를 촉진하는 강력한 메커니즘입니다.
- **자동화 및 재해 복구:** IaC는 인프라 생성 및 변경 프로세스를 완전히 자동화하여 수동 작업으로 인한 지연과 오류를 제거합니다. 또한, 재해 발생 시 코드를 재실행하는 것만으로 전체 인프라를 신속하고 정확하게 복구할 수 있습니다.

WIF 구성에 IaC를 도입하는 것은 단순한 자동화 이상의 의미를 지닙니다. 이는 보안 및 운영 규율의 근본적인 전환을 의미하며, `invalid_target` 오류를 유발했던 설정 불일치와 같은 문제를 구조적으로 예방하는 가장 효과적인 방법입니다.

다음 표는 수동 구성 방식과 IaC 접근법의 핵심적인 차이점을 명확하게 보여줍니다.

기능	수동 구성 (콘솔/gcloud)	IaC 구성 (Terraform)
재현성	낮음 (절차에 따라 결과가 다를 수 있음)	높음 (코드가 동일한 결과를 보장)
감사 가능성	낮음 (Cloud Audit Log에 의존, 컨텍스트 파악 어려움)	높음 (Git 이력을 통해 모든 변경의 컨텍스트 추적 가능)
버전 관리	불가능	가능 (Git을 통한 완전한 변경 이력 관리)
동료 검토	불가능	가능 (Pull Request를 통한 사전 검증 및 협업)

재해 복구	느리고 오류 발생 가능성 높음 (수동 재구성 필요)	빠르고 신뢰성 높음 (코드 재실행으로 자동 복구)
보안 (설정 오류 위험)	높음 (인적 오류, 설정 불일치 발생 용이)	낮음 (terraform plan을 통한 사전 검토, 자동화된 적용)

이 표는 IaC 도입이 단순한 기술적 선택이 아니라, 비즈니스의 안정성, 보안, 그리고 효율성을 담보하는 필수적인 전략적 투자임을 명확히 보여줍니다.

3.2. WIF 구현을 위한 핵심 Terraform 리소스

Terraform을 사용하여 Google Cloud에서 WIF를 구성하기 위해서는 다음 세 가지 핵심 리소스를 이해하고 활용해야 합니다.

1. **google_iam_workload_identity_pool**: 외부 ID 공급자를 그룹화하는 최상위 컨테이너인 '워크로드 아이덴티티 풀'을 생성합니다. 일반적으로 조직 또는 환경 단위로 하나의 풀을 생성합니다.¹⁶
2. **google_iam_workload_identity_pool_provider**: 풀 내부에 특정 외부 ID 공급자를 구성합니다. GitHub Actions의 경우 OIDC 공급자로 생성되며, issuer_uri (발급자 URL), attribute_mapping (속성 매핑), attribute_condition (속성 조건)과 같은 핵심 파라미터를 정의합니다.¹⁷
3. **google_service_account_iam_binding** 또는 **google_service_account_iam_member**: 특정 외부 ID가 서비스 계정을 위장할 수 있도록 roles/iam.workloadIdentityUser 역할을 서비스 계정에 바인딩합니다. 이 리소스가 WIF의 인증과 GCP의 승인을 연결하는 다리 역할을 합니다.¹⁹

이 중 가장 중요하고 섬세한 설정이 필요한 부분은 IAM 바인딩 리소스의 member (또는 members) 인수입니다. WIF의 경우, 이 값은 단순한 이메일 주소가 아니라 principalSet이라는 특별한 형식의 문자열을 사용합니다.¹ 이 형식은 OIDC 토큰에서 매핑된 속성을 기반으로 접근 권한을 부여할 수 있게 해주며, WIF의 세분화된 접근 제어 능력을 코드로 구현하는 핵심 요소입니다.

Terraform은 이러한 복잡한 principalSet 문자열을 동적으로, 그리고 오류 없이 생성할 수 있는 강력한 기능을 제공합니다. 예를 들어, 특정 GitHub 리포지토리의 모든 브랜치에서 오는 요청을 허용하는 principalSet은 다음과 같이 코드로 표현할 수 있습니다.

Terraform


```
resource "google_service_account_iam_member" "wif_binding" {
  service_account_id = google_service_account.gha_deploy_sa.name
  role                = "roles/iam.workloadIdentityUser"

  # 'principalSet' 형식을 사용하여 WIF 주체를 지정
  member =
    "principalSet://iam.googleapis.com/${google_iam_workload_identity_pool.github_pool.name}/
    attribute.repository/${var.github_repository}"
}
```

이 코드는 `google_iam_workload_identity_pool.github_pool.name`을 통해 풀의 전체 리소스 이름을 동적으로 참조하고, 변수 `var.github_repository`를 사용하여 리포지토리 경로를 주입합니다. 이러한 방식은 하드코딩으로 인한 실수를 방지하고, 코드를 재사용 가능하게 만들어 WIF 구성의 유지보수성과 신뢰성을 크게 향상시킵니다. 이는 기존 보고서에서 제시된 `principalSet` 경로¹를 실제 코드¹⁹로 구현하여 추상적인 개념을 구체적이고 재사용 가능한 패턴으로 전환하는 과정을 보여줍니다.

3.3. Terraform을 통한 보안 모범 사례 구현

Google Cloud의 WIF 보안 모범 사례는 GitHub와 같이 여러 조직이 함께 사용하는 다중 테넌트(multi-tenant) ID 공급자를 연동할 때, 매우 제한적인 `attribute_condition` 표현식을 사용할 것을 강력히 권고합니다.⁷ 이 조건은 OIDC 토큰의 클레임 값을 검사하여, 의도된 조직, 리포지토리, 브랜치에서 발급된 토큰이 아닐 경우 토큰 교환 자체를 거부하는 역할을 합니다. 이는 WIF 보안의 최전선 방어선입니다.

`attribute_condition`은 WIF 환경에서 '혼란스러운 대리인(confused deputy)' 공격을 방지하는 핵심적인 보안 통제 수단입니다. 만약 이 조건이 없다면, 여러분의 WIF 공급자 이름을 아는 GitHub의 다른 어떤 조직이나 사용자가 자신의 워크플로에서 여러분의 GCP 프로젝트에 인증을 시도할 수 있는 잠재적 위험이 존재합니다 (물론 IAM 바인딩이 허용적으로 설정되었다는 가정 하에). Terraform을 사용하여 엄격한 조건을 코드로 명시함으로써, 이러한 보안 통제를 인프라의 필수적이고 감사 가능한 일부로 만들 수 있습니다.

다음은 `quali-journal` 리포지토리의 `main` 브랜치에서 실행되는 워크플로만 허용하도록 `attribute_condition`을 설정하는 Terraform 코드 예시입니다.

Terraform

```
resource "google_iam_workload_identity_pool_provider" "github_actions_provider" {
```



```

project          = var.project_id
workload_identity_pool_id      =
google_iam_workload_identity_pool.github_pool.workload_identity_pool_id
workload_identity_pool_provider_id = "gha-provider"
display_name     = "Quali Journal GitHub Actions Provider"

oidc {
  issuer_uri = "https://token.actions.githubusercontent.com"
}

# OIDC 토큰의 클레임을 GCP 속성으로 매핑
attribute_mapping = {
  "google.subject"    = "assertion.sub"
  "attribute.actor"   = "assertion.actor"
  "attribute.repository" = "assertion.repository"
  "attribute.ref"     = "assertion.ref"
}

# 토큰 교환을 허용할 조건을 CEL(Common Expression Language)로 정의
attribute_condition = "attribute.repository == 'cdmacs1003-cyber/quali-journal' &&
attribute.ref == 'refs/heads/main'"
}

```

기존 보고서는 principalSet 바인딩을 통해 접근 범위를 제한하는 방법을 언급했지만¹, 공급자 수준에서 attribute_condition을 사용하는 것이 더 강력하고 중앙 집중적인 통제 방식입니다.⁸ 이 조건은 IAM 정책이 평가되기도 전인 가장 이른 단계에서 유효하지 않은 토큰을 거부합니다. 이처럼 공급자 수준의 조건과 IAM principalSet 바인딩을 함께 사용하는 계층적 보안(layered security) 접근법은 성숙한 클라우드 보안 아키텍처의 특징이며, Terraform은 이러한 고급 보안 패턴을 쉽고 안정적으로 구현하고 유지 관리할 수 있도록 지원합니다.

섹션 4: IAM 심층 분석: Cloud Run 배포를 위한 최소 권한 원칙 적용

본 섹션에서는 CI/CD 파이프라인에 필요한 권한을 정밀하게 분석하고, 이를 바탕으로 불필요한 권한을 모두 제거한 최소 권한의 맞춤형 IAM 역할을 설계 및 구현하는 구체적인 방법을 제시합니다.

4.1. 배포용 사전 정의 역할 해부

기존 보고서는 CI/CD 서비스 계정에 `roles/run.admin`, `roles/cloudbuild.builds.editor`, `roles/artifactregistry.writer`, `roles/secretmanager.secretAccessor`와 같은 여러 사전 정의된 역할을 부여할 것을 제안했습니다.¹ 이 역할들은 편리하지만, 일반적인 애플리케이션 배포 워크플로의 요구사항을 훨씬 초과하는 광범위한 권한을 포함하고 있어 보안상 심각한 위험을 초래할 수 있습니다.

각 역할에 포함된 권한을 상세히 분석해 보면 그 위험성이 명확해집니다:

- **roles/run.admin:** 이 역할은 Cloud Run 서비스를 생성하고 업데이트하는 권한 외에도, 서비스를 삭제(`run.services.delete`)하고 서비스의 IAM 정책을 수정(`run.services.setIamPolicy`)할 수 있는 매우 강력한 권한을 포함합니다.²² 배포 자동화 주체에게 서비스 삭제나 권한 변경 능력을 부여하는 것은 매우 위험합니다.
- **roles/cloudbuild.builds.editor:** 빌드를 생성하는 권한뿐만 아니라, 프로젝트 내의 다른 모든 빌드를 취소하거나 수정(`cloudbuild.builds.update`)할 수 있는 권한을 부여합니다.²³ 이는 해당 서비스 계정이 다른 팀의 빌드 프로세스를 방해할 수 있는 가능성을 열어둡니다.
- **roles/artifactregistry.writer:** 아티팩트를 업로드하는 권한과 함께, 기존 아티팩트를 덮어쓰거나 삭제할 수 있는 권한을 포함합니다.²⁵ 배포 전용 서비스 계정이 프로덕션 환경의 안정적인 아티팩트를 삭제할 수 있게 하는 것은 잠재적인 서비스 장애의 원인이 될 수 있습니다.

이처럼 광범위한 사전 정의 역할을 사용하는 것은 "공격 표면(attack surface)"과 "폭발 반경(blast radius)"을 불필요하게 확대하는 결과를 낳습니다. 만약 이 배포 서비스 계정의 자격 증명이 어떤 경로로든 유출된다면, 공격자는 부여된 과도한 권한을 악용하여 서비스를 중단시키거나(Cloud Run 서비스 삭제), 다른 리소스에 접근하기 위해 권한을 상승시키는(IAM 정책 수정) 등의 심각한 피해를 야기할 수 있습니다. 최소 권한 원칙은 단순히 미승인 접근을 방지하는 것을 넘어, 합법적인 자격 증명에 침해되었을 때 발생할 수 있는 잠재적 피해를 최소화하는 데 그 핵심적인 목적이 있습니다.

배포 자동화의 목적은 애플리케이션의 새로운 버전을 '배포'하는 것이지, 관련 서비스를 '관리'하는 것이 아닙니다. 이 둘의 차이를 명확히 인식하는 것이 중요합니다. 배포 프로세스는 빌드를 '생성'하고, 아티팩트를 '업로드'하며, 서비스를 '업데이트'하는 작업이 필요합니다. 서비스를 '삭제'하거나, IAM 정책을 '설정'하거나, 다른 빌드를 '취소'할 필요는 없습니다. CI/CD 워크플로에서 실행되는 각 `gcloud` 명령어가 어떤 GCP API를 호출하고, 해당 API가 어떤 IAM 권한을 요구하는지 정확히 매핑하는 분석 과정을 통해, 사전 정의된 역할이 얼마나 과도한지 증명하고 안전한 맞춤형 역할을 설계할 수 있는 기반을 마련할 수 있습니다.²⁷

4.2. 최소 권한 맞춤형 IAM 역할 구성

최소 권한 원칙을 실제로 구현하는 가장 효과적인 방법은 필요한 권한만을 정확히 담은 맞춤형 IAM

역할을 생성하는 것입니다.²⁸ 이 과정은 다음과 같은 체계적인 단계를 통해 이루어집니다:

1. **필요 작업 식별:** CI/CD 워크플로의 모든 단계를 나열합니다 (예: 소스 코드 다운로드, 컨테이너 빌드, 아티팩트 푸시, Cloud Run 배포, 시크릿 접근).
2. **권한 매핑:** 각 작업에 필요한 최소한의 IAM 권한을 GCP 공식 문서를 참조하여 정확히 찾아냅니다.
3. **맞춤형 역할 정의:** 찾아낸 권한 목록을 사용하여 Terraform의 `google_project_iam_custom_role` 리소스나 YAML 파일을 통해 맞춤형 역할을 정의합니다.

다음 표는 일반적인 Cloud Run 배포 워크플로를 기반으로 필요한 IAM 권한을 분석한 예시이며, 이는 맞춤형 역할을 설계하기 위한 청사진 역할을 합니다.

워크플로 단계	필요 작업	GCP 서비스	최소 IAM 권한	정당성
gcloud builds submit	컨테이너 이미지 빌드 작업 제출	Cloud Build	cloudbuild.buids.create	CI/CD 파이프라인이 Cloud Build 서비스를 호출하여 빌드를 시작하기 위해 필수적인 권한입니다.
빌드 중 이미지 푸시	빌드된 컨테이너 이미지를 레지스트리에 업로드	Artifact Registry	artifactregistry.repositories.uploadArtifacts	Cloud Build 작업이 결과물인 컨테이너 이미지를 지정된 Artifact Registry 리포지토리에 저장하기 위해 필요합니다.
gcloud run deploy	Cloud Run 서비스의 새 리비전(revision) 배포	Cloud Run	run.services.update	기존 Cloud Run 서비스를 새로운 컨테이너 이미지로 업데이트하고 트래픽을 전환하기 위한 핵심 권한입니다.

gcloud run deploy	Cloud Run 서비스가 새 이미지를 가져오도록 허용	IAM	iam.serviceAccounts.actAs	(매우 중요) 배포 프로세스가 Cloud Run 서비스의 런타임 서비스 계정을 위장(impersonate)하여 새 리비전을 생성할 수 있도록 허용하는 권한입니다. 이 권한이 없으면 Cloud Run 서비스는 배포를 수행할 권한을 얻지 못해 실패합니다. ³¹
빌드/배포 중 시크릿 접근	빌드 시점의 환경 변수나 API 키 등 민감 정보 접근	Secret Manager	secretmanager.versions.access	빌드 과정이나 애플리케이션 런타임에 필요한 시크릿 값을 안전하게 가져오기 위해 필요합니다. ³³

이 분석을 바탕으로, Terraform을 사용하여 최소 권한의 맞춤형 역할을 다음과 같이 코드로 정의할 수 있습니다.

Terraform

```
resource "google_project_iam_custom_role" "cloud_run_deployer" {
  project = var.project_id
  role_id = "cloudRunDeployer"
  title   = "Cloud Run Deployer"
  description = "Minimal permissions required to deploy a new version to Cloud Run via GitHub Actions."

  permissions = [
    "cloudbuild.builds.create",
```

```

"artifactregistry.repositories.uploadArtifacts",
"run.services.update",
"iam.serviceAccounts.actAs",
"secretmanager.versions.access",
# 리소스 상태 조회를 위한 읽기 권한 추가
"run.services.get",
"artifactregistry.repositories.get"
]
}

# 생성된 맞춤형 역할을 배포용 서비스 계정에 부여
resource "google_project_iam_binding" "deployer_binding" {
  project = var.project_id
  role    = google_project_iam_custom_role.cloud_run_deployer.name

  members = [
    "serviceAccount:${google_service_account.gha_deploy_sa.email}"
  ]
}

```

이 접근 방식은 추상적인 보안 원칙을 구체적이고 실행 가능한 코드로 전환합니다. 특히, 복잡한 배포 시나리오에서 종종 문제를 일으키는 iam.serviceAccounts.actAs 권한의 필요성과 역할을 명확히 함으로써, 안정적이고 안전한 배포 자동화를 구현할 수 있는 실질적인 가이드를 제공합니다.

섹션 5: 종합 및 전략적 권고

본 보고서의 종합적인 분석을 바탕으로, 'Quali Journal' 프로젝트가 현재의 인증 문제를 해결하고 장기적으로 안전하며 효율적인 CI/CD 아키텍처를 구축하기 위한 구체적이고 우선순위가 정해진 실행 계획을 제시합니다.

5.1. 즉각적인 실행 계획 (향후 48시간 내)

이 계획은 현재 발생 중인 배포 차단 문제를 가장 빠르고 효율적으로 해결하는 데 초점을 맞춥니다. 추측에 기반한 반복적인 시도를 지양하고, 데이터 기반의 정밀 진단을 통해 문제의 근본 원인을 신속하게 제거하는 것을 목표로 합니다.

1. **진단 가시성 확보:** 즉시 Google Cloud 프로젝트의 IAM & Admin 섹션에서 **Security Token**

Service API에 대한 데이터 액세스(Data Access) 감사 로그('Admin Read', 'Data Read')를 활성화하십시오.¹⁰ 이는 모든 토큰 교환 시도를 기록하여 실패 원인을 명확히 파악할 수 있는 기반이 됩니다.

2. **오류 데이터 수집:** 감사 로그가 활성화된 상태에서, 실패하는 GitHub Actions 워크플로를 다시 한번 실행하여 `invalid_target` 오류를 재현합니다. 이를 통해 상세한 오류 정보가 포함된 로그 항목이 생성됩니다.
3. **근본 원인 분석 및 수정:** GCP 콘솔의 Logs Explorer를 사용하여 위 단계에서 생성된 오류 로그를 분석합니다. 로그의 `protoPayload`에 포함된 구체적인 오류 메시지, `resourceName`, `principalSubject` 등의 정보를 통해 잘못 설정된 항목(예: GitHub 시크릿에 저장된 프로젝트 번호, 워크플로 파일의 공급자 경로 오타 등)을 정확히 식별합니다.
4. **검증:** 식별된 단일 구성 오류를 수정한 후, 워크플로를 다시 실행하여 WIF 인증이 성공적으로 완료되는지 확인합니다.

이 계획은 느린 시행착오 접근법을 피하고 고급 진단 도구를 활용하여 핵심 문제를 직접적으로 해결함으로써, 개발팀의 생산성 저하를 최소화하고 신속하게 배포 파이프라인을 정상화할 수 있도록 합니다.

5.2. 중기 전략 (향후 90일 내)

즉각적인 문제가 해결된 후에는, 동일한 문제가 재발하지 않도록 구조적인 개선을 추진해야 합니다. 이 중기 전략은 수동 구성의 취약점을 제거하고, 보안 부채를 청산하며, 자동화되고 감사 가능한 IaC(Infrastructure as Code) 기반의 관리 체계로 전환하는 것을 목표로 합니다.

1. **IaC 리포지토리 구축:** GCP 인프라 관리를 위한 전용 Git 리포지토리를 생성하고 Terraform 실행 환경을 구성합니다.
2. **WIF 구성 코드화:** 본 보고서의 섹션 3에서 제시된 Terraform 코드를 사용하여 WIF 풀, 공급자, 그리고 모든 관련 IAM 바인딩을 코드로 정의하고 적용합니다. 특히, 리포지토리 및 브랜치를 제한하는 엄격한 `attribute_condition`을 포함하여 보안을 강화합니다.
3. **최소 권한 역할 구현:** 섹션 4에서 설계한 최소 권한의 맞춤형 IAM 역할을 `google_project_iam_custom_role` Terraform 리소스를 사용하여 생성합니다.
4. **권한 재구성 및 적용:** 기존에 배포 서비스 계정(`gha-deploy-sa`)에 부여되었던 과도한 권한의 사전 정의 역할들(`roles/run.admin` 등)을 모두 제거하고, 새로 생성한 맞춤형 역할 하나만을 부여하도록 Terraform 코드를 수정하여 적용합니다.
5. **보안 부채 청산:** 만약 임시방편으로 서비스 계정 키를 사용했다면, 이 시점에서 GCP 콘솔에서 해당 키를 즉시 비활성화하고 삭제합니다. 동시에, GitHub Actions에서 `GCP_SA_KEY` 시크릿을 완전히 제거하여 완전한 키리스(keyless) 인증 체계로의 전환을 완료합니다.¹

이 로드맵은 보안 취약점과 운영상의 불안정성을 체계적으로 제거하고, 프로젝트를 안전하고 자동화되었으며 감사 가능한 최신 클라우드 관리 방식으로 전환하는 구체적인 경로를 제시합니다.

5.3. 장기적 비전: 안전한 CI/CD 보안 태세

본 보고서에서 제시한 로드맵을 성공적으로 완수함으로써, 'Quali Journal' 프로젝트는 수동 구성에 의존하는 취약한 CI/CD 인증 시스템에서 벗어나, 업계 최고 수준의 안전한 보안 태세를 갖춘 아키텍처로 진화하게 될 것입니다.

이러한 전환을 통해 달성되는 핵심적인 성과는 다음과 같습니다:

- **제로 트러스트 기반의 키리스 인증:** CI/CD 파이프라인에서 수명이 긴 정적 서비스 계정 키를 완전히 제거하여, 자격 증명 유출로 인한 핵심적인 보안 위협을 원천적으로 차단합니다.
- **최소 권한 원칙의 완전한 구현:** 배포 주체는 자신의 기능 수행에 필수적인 최소한의 권한만을 가지게 되어, 만일의 침해 사고 발생 시에도 피해 범위를 극도로 제한(minimize blast radius)할 수 있습니다.
- **감사 가능하고 버전 관리되는 ID 관리:** 모든 인증 및 승인 정책의 변경 사항은 GitOps 워크플로를 통해 추적, 검토, 승인되므로, 완전한 투명성과 통제력을 확보하게 됩니다.
- **향상된 보안 및 규정 준수:** 결과적으로 구축된 아키텍처는 외부 공격에 대해 훨씬 더 높은 복원력을 갖추게 되며, 각종 보안 규정 준수(compliance) 요구사항을 증명하는 과정을 단순화하고 신뢰도를 높입니다.

궁극적으로, 이는 단순히 기술적인 문제를 해결하는 것을 넘어, 개발팀이 인프라 걱정 없이 비즈니스 가치 창출에 집중할 수 있도록 하는 안정적이고 신뢰할 수 있는 플랫폼을 구축하는 과정입니다.

참고 자료

1. 1018_1WIF 인증 문제 해결.pdf
2. Who Touched My GCP Project? Understanding the Principal Part in Cloud Audit Logs – Part 2 - Mitiga, 10월 18, 2025에 액세스,
<https://www.mitiga.io/blog/who-touched-my-gcp-project-understanding-the-principal-part-in-cloud-audit-logs-part-2>
3. GKE Workload Identity return 400 when accessing Google Cloud resources, 10월 18, 2025에 액세스,
<https://discuss.google.dev/t/gke-workload-identity-return-400-when-accessing-google-cloud-resources/173230>
4. Securing Your CI/CD Pipeline: Eliminate Long-Lived Credentials with Workload Identity Federation (2) | Community, 10월 18, 2025에 액세스,
<https://security.googlecloudcommunity.com/community-blog-42/securing-your-ci-cd-pipeline-eliminate-long-lived-credentials-with-workload-identity-federation-2-3909>
5. Terraform module for workload identity federation on GCP. - GitHub, 10월 18, 2025에 액세스,
<https://github.com/mscribellito/terraform-google-workload-identity-federation>
6. Authenticate to Google Cloud from Terraform Cloud using workload identity -

- Astrafy, 10월 18, 2025에 액세스,
<https://astrafy.io/the-hub/blog/technical/authenticate-to-google-cloud-from-terraform-cloud-using-workload-identity>
7. Best practices for using Workload Identity Federation | IAM ..., 10월 18, 2025에 액세스,
<https://cloud.google.com/iam/docs/best-practices-for-using-workload-identity-federation>
 8. Use dynamic credentials with the GCP provider in Terraform Enterprise, 10월 18, 2025에 액세스,
<https://developer.hashicorp.com/terraform/enterprise/workspaces/dynamic-provider-credentials/gcp-configuration>
 9. Access Google Cloud from HCP Terraform with workload identity - HashiCorp, 10월 18, 2025에 액세스,
<https://www.hashicorp.com/blog/access-google-cloud-from-hcp-terraform-with-workload-identity>
 10. Example logs for Workload Identity Federation | Identity and Access ..., 10월 18, 2025에 액세스,
<https://cloud.google.com/iam/docs/audit-logging/examples-workload-identity>
 11. Troubleshoot Workload Identity Federation | IAM Documentation - Google Cloud, 10월 18, 2025에 액세스,
<https://cloud.google.com/iam/docs/troubleshooting-workload-identity-federation>
 12. Troubleshoot IAM permissions | Policy Intelligence - Google Cloud, 10월 18, 2025에 액세스,
<https://cloud.google.com/policy-intelligence/docs/troubleshoot-access>
 13. Troubleshoot policies | Identity and Access Management (IAM) - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/troubleshoot-policies>
 14. How to use Policy Troubleshooter - YouTube, 10월 18, 2025에 액세스,
<https://www.youtube.com/watch?v=geqSzNvZOuk>
 15. Authentication for Terraform - Google Cloud, 10월 18, 2025에 액세스,
<https://cloud.google.com/docs/terraform/authentication>
 16. google_iam_workload_identity_, 10월 18, 2025에 액세스,
https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/iam_workload_identity_pool
 17. google_iam_workload_identity_pool_provider | Resources ..., 10월 18, 2025에 액세스,
https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/iam_workload_identity_pool_provider
 18. Google Cloud IAM IAM Workload Identity Pool Provider - Examples and best practices, 10월 18, 2025에 액세스,
https://shisho.dev/dojo/providers/google/Cloud_IAM/google-iam-workload-identity-pool-provider/
 19. How do I create the workload identity IAM bindings in terraform ..., 10월 18, 2025에 액세스,
<https://discuss.hashicorp.com/t/how-do-i-create-the-workload-identity-iam-bindings-in-terraform/19501>

20. gcp.serviceaccount.IAMPolicy | Pulumi Registry, 10월 18, 2025에 액세스, <https://www.pulumi.com/registry/packages/gcp/api-docs/serviceaccount/iampolicy/>
21. Configuring GCP Workload Identity Federation For GitHub Actions - Jon Winton, 10월 18, 2025에 액세스, <https://jonwinton.com/posts/2024/08/configuring-gcp-workload-identity-federation-for-github-actions/>
22. Cloud Run IAM roles | Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/run/docs/reference/iam/roles>
23. IAM roles and permissions | Cloud Build, 10월 18, 2025에 액세스, <https://cloud.google.com/build/docs/iam-roles-permissions>
24. Cloud Build roles and permissions | Identity and Access Management (IAM) - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/roles-permissions/cloudbuild>
25. Access control with IAM | Artifact Registry - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/artifact-registry/docs/access-control>
26. Artifact Registry roles and permissions | Identity and Access Management (IAM) | Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/roles-permissions/artifactregistry>
27. I feel like I finally figured out IAM permissions : r/googlecloud - Reddit, 10월 18, 2025에 액세스, https://www.reddit.com/r/googlecloud/comments/1ga44kw/i_feel_like_i_finally_figured_out_iam_permissions/
28. IAM Custom Roles | Google Cloud Skills Boost, 10월 18, 2025에 액세스, <https://www.cloudskillsboost.google/focuses/1035?parent=catalog>
29. Roles and permissions | Identity and Access Management (IAM) - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/roles-overview>
30. google_project_iam_custom_role | Resources | hashicorp/google - Terraform Registry, 10월 18, 2025에 액세스, https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_project_iam_custom_role
31. Roles for service account authentication | Identity and Access Management (IAM), 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/service-account-permissions>
32. Compute Engine IAM roles and permissions - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/compute/docs/access/iam>
33. Access control with IAM | Secret Manager - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/secret-manager/docs/access-control>
34. Secret Manager roles and permissions - IAM - Google Cloud, 10월 18, 2025에 액세스, <https://cloud.google.com/iam/docs/roles-permissions/secretmanager>